The design of `GaugeTheory/`

Jinzheng Li

First draft 22 September 2026; rewritten 3 November 2026; field data and morphisms, 9 November 2026

What the library must do

The target is one theorem (§1): for the Standard Model, the gauge- and Lorentz-invariant local expressions of mass dimension at most four are spanned by an explicit list. Reaching it in a way any other model can reuse needs four things. Status as of 9 November 2026.

1. **Describe a theory from its card.** A gauge group given as a list of factors and a table of fields with charges and generations produce everything else: the gauge data, the matter fields, the connection, and the field data of the theory. (*done; the field data exists as* `GaugeFieldData`*, to be renamed; §3.1–3.3, §4*)
2. **Write and classify its local expressions.** The algebra of the symbols $\partial_s \psi_\alpha$ carries the gauge and Lorentz actions, the total derivative and the mass-weight grading; a term written from the card's names can be checked for invariance, and the invariants of a given mass dimension come as a spanning list, one family of terms at a time. (*done on hand-built data for the Standard Model; the generic statement and the card-level interface to do; §3.5, §4, §6*)
3. **Relate theories to one another.** Two descriptions of a theory can be shown equivalent, a theory embedded in or projected onto another, or restricted along a change of gauge group, and every construction above respects these maps; sectors are the parts belonging to sub-theories. (*designed; the first piece implemented, PR coming; §3.4*)
4. **Reality, and what comes after.** Real fields and a star, so that a real Lagrangian is a star-fixed invariant; then invariants modulo total derivatives and equations of motion, symmetry breaking, flavour symmetries and higher mass dimension. (*to do; the rest after the deadline; §3.5, §4, §6*)

1 Purpose

`PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/` is the symbolic framework for the local content of a gauge theory: the algebra in which a Lagrangian is written, the gauge and Lorentz actions on it, and the invariants of a given mass dimension. Fields enter only through their jets at one point, the formal symbols $\partial_s \psi_\alpha$.

One rule organises the folder: **every layer is parametrised by data and returns constructions and theorems; a concrete theory contributes only its data.** A model supplies a gauge datum and a table of matter fields, and everything else about it is inherited.

The target theorem. For the Standard Model, the gauge- and Lorentz-invariant elements of the local field algebra of mass weight at most 8 (mass dimension at most 4) are spanned by an explicit list. This is a classification of invariant *polynomials*: it is over $\mathbb{C}$, imposes no reality condition, and

takes no quotient by total derivatives or equations of motion. It therefore contains terms such as $\partial^2(H^\dagger H)$ and $\operatorname{tr} F\tilde{F}$, and it should be named as a statement about invariants, not as "the Lagrangian". Reality and the quotient by total derivatives are later steps (§6).

Deadline. Early December 2026, with the end of the semester as the hard deadline.

2 The idea

- **Gauge jets.** A gauge transformation is its jet U in a jet group G_J: value $\operatorname{eval} U \in G_0$, Lie algebra $\mathfrak{g}_J$, derivatives, adjoint action, Maurer–Cartan form $\omega_\mu(U) = i(\partial_\mu U)U^{-1}$. These maps and their laws are the gauge datum; nothing downstream knows which group it is.

- **Matter fields.** A matter field is a finite-dimensional complex space V with a Lorentz representation, an action of G_J on $\texttt{JetRing} \otimes V$ that is fibrewise and generated by an infinitesimal action of $\mathfrak{g}$, a mass weight and a statistic (bosonic or fermionic). Its symbols transform linearly.

- **The gauge field.** The connection is not listed by a theory: it is determined by the gauge datum. Its symbols transform affinely, by the adjoint action plus the Maurer–Cartan form, and the translation part satisfies the cocycle law $t(UV) = t(U) + \operatorname{Ad}_U t(V)$. Matter and the connection are two separate notions; the affine picture describes how they differ but is not a common type.

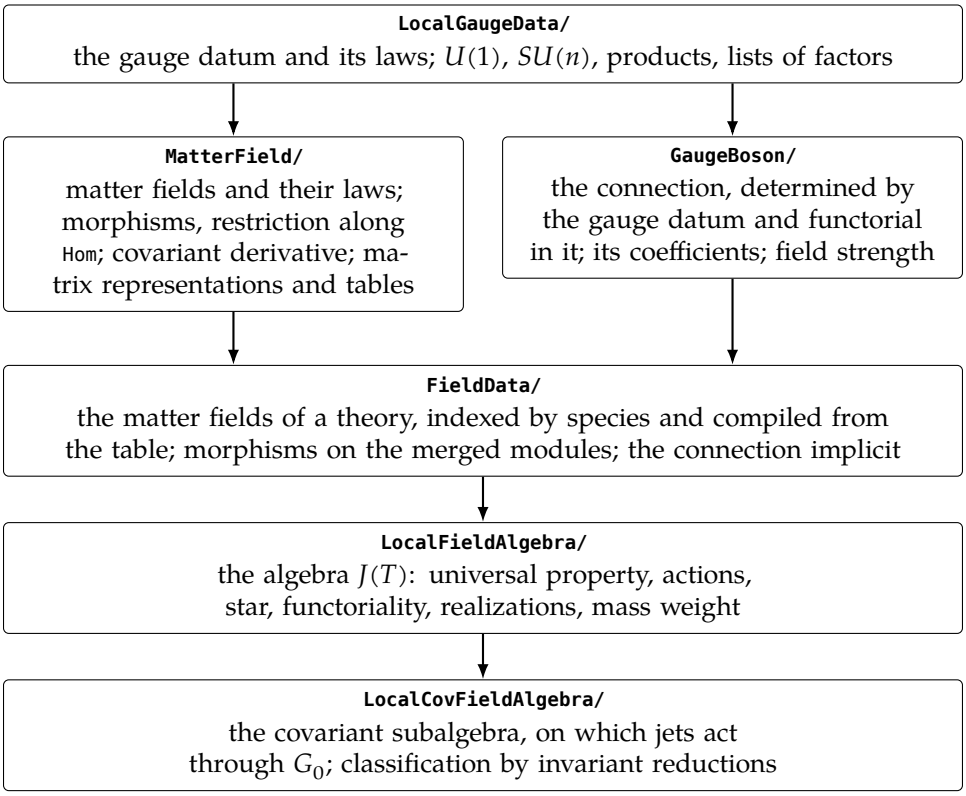
- **Field data.** A theory's field data is a finite family of matter fields indexed by its species, bosonic and fermionic kept apart, compiled from its table with one species per multiplet and generation; it exists as `GaugeFieldData` and is to be renamed `FieldData`. The family is a *presentation*: what a theory is, is its merged bosonic and fermionic modules with their actions, and morphisms see only these. The connection is added from the gauge datum, so the pure gauge theory is the field data with no species, and the sum of two theories is the disjoint union of their species.

- **Morphisms.** Equivalence, embedding and projection of models are morphisms, never equality. Over one gauge datum a morphism is an intertwiner of the merged modules, equivalently a matrix of intertwiners between species, not a bijection of species, because the split into species is not canonical when a multiplet repeats. Along a morphism of gauge data, matter fields restrict and the connection maps forward. The connection has no free data, so it transports automatically; the branching of a gauge sector (the X, Y bosons of $SU(5)$ seen from the Standard Model) is a statement about the algebras, not about the field data.

- **The algebra.** $J(T)$ is the free graded-commutative algebra on the symbols of the matter fields of T, their conjugates, and the connection. Every action is defined on generators and extended by the universal property; the translation part of the connection gives the inhomogeneous term. A morphism of field data gives an equivariant map of algebras in the opposite direction, the symbols being covectors, and sectors are images of such maps.

- **Invariants.** Mass weight is twice the mass dimension (derivative 2, scalar 2, Weyl fermion 3, connection 2). On the covariant subalgebra jets act through G_0, so classification is ordinary invariant theory. It is done one family of terms at a time: a *reduction* ReducesInvariantsTo $\sigma\ V\ W$ says that for every σ-stable submodule S, an invariant of $V \sqcup S$ lies in $W \sqcup S$. Reductions compose, combine over finite joins and transport along maps of index sets, and σ is any family of linear maps, so gauge and Lorentz invariance are one invariance for the disjoint union of the two groups.

3 The layers



Imports go down the line only. Three points decide the shape:

- **Two field notions, one interface per model.** Matter fields and the connection are separate: each states its own transformation laws once, and the matter laws are stated for an arbitrary `MatterField`. A statement over all fields of a theory uses a type built once the model is fixed,

```
inductive Field (T : FieldData jets)
  | boson (i : T.BosonSpecies) | fermion (i : T.FermionSpecies) | gauge
```

  which lives in `Type`, has decidable equality, and whose laws dispatch to the matter and connection laws. A closed type `Species | matter M | gauge` at the level of data was tried (PR #2011) and dropped: every general lemma split into the two cases, the matter case only restated existing results, and it did not say how a theory contains its gauge field.

- **One algebra tower.** The pure gauge theory is J of the field data with no matter, and the sectors of $J(T)$ are images of the maps induced by projections onto sub-theories (§3.4). Nothing is built twice and identified afterwards.

- **One classification mechanism.** The relation `ReducesInvariantsTo` of `Mathematics/InvariantReduction` is general mathematics and stays there. The algebra layer supplies what every theory needs to use it: the gauge and Lorentz actions as one family indexed by $G_0 \sqcup SL(2,\mathbb{C})$, the reduction of a factor's invariance to that family, and the stability of symbol ranges, all in `LocalCovFieldAlgebra/`. A classification theorem for one index law (a colour trace, an isospin contraction, a Lorentz scalar) is stated as a reduction modulo an arbitrary stable submodule, so it can be applied inside any larger expression and to any extension.

Each layer is described below by its datum, what it derives and its interface.

3.1 `LocalGaugeData/`: the gauge datum

The question it answers. A Lagrangian density at a point x depends on the fields and finitely many of their derivatives at x. So when we ask whether it is gauge invariant, a gauge transformation $g(x)$ matters only through its Taylor series at that point, its *jet*. This layer describes these jets and nothing else. It does not mention fields or Lagrangians; every later layer takes its datum as input.

An example first. For $U(1)$, a gauge transformation is $u(x) = e^{i\theta(x)}$. Its jet is the power series of u in the coordinates $x_0, \dots, x_3$, a unitary element of $\mathbb{C}[[x]]$. Its value at the base point is $u(0) \in U(1)$. Its *Maurer–Cartan form* $\omega_\mu = i(\partial_\mu u)u^{-1} = -\partial_\mu\theta$ is exactly the shift of the gauge field, $A_\mu \mapsto A_\mu + \omega_\mu$. For $SU(n)$ everything is the same with $n \times n$ matrices of power series, and $\omega_\mu = i(\partial_\mu U)U^{-1}$ is a traceless hermitian matrix.

The datum. `LocalGaugeData` G_0 $\mathfrak{g}$ `GJ` $\mathfrak{g}_J$ abstracts the example. It has four types: the gauge group G_0 and its Lie algebra $\mathfrak{g}$, and their jet versions G_J and $\mathfrak{g}_J$. It has the maps a transformation law uses:

- evaluation at the base point and constant jets, $G_J \leftrightarrows G_0$ and $\mathfrak{g}_J \leftrightarrows \mathfrak{g}$;
- derivatives ∂_μ and multiplication by coordinates x_μ on $\mathfrak{g}_J$;
- the adjoint action Ad_U of a gauge jet on Lie algebra jets;
- the Maurer–Cartan form $\omega_\mu(U)$.

It also has the rules these satisfy in the example, such as $\omega(UV) = \omega(U) + \mathrm{Ad}_U\,\omega(V)$ and $\partial_\mu\omega_\nu - \partial_\nu\omega_\mu + [\omega_\mu, \omega_\nu] = 0$. Everything later is proved from these rules alone, so it holds for every gauge group at once.

What is proved from the rules. Three tools that later layers need:

- how Ad_U acts on Taylor coefficients (a Leibniz formula), which is how a gauge jet moves the derivatives of a field;
- the *symmetrised* Maurer–Cartan data, a smaller set of numbers from which all Taylor coefficients of ω can be recovered;
- the jets that agree with the identity up to order n; these form normal subgroups, and such a jet only shifts the first n derivatives of the gauge field.

Two extra properties. The rules do not say that G_J really consists of Taylor series, so two properties are stated separately. `Faithful`: a jet is determined by its Taylor coefficients. `Free`: every choice of Taylor coefficients comes from some jet. `Free` is what later shows that the gauge field enters an invariant only through its field strength: gauge jets are rich enough to remove every other combination of derivatives of A_μ.

Where concrete groups come from. A *matrix presentation* (`MatrixJets`) says that the four types sit inside matrices and every map is the matrix one. From it the rules and `Faithful` are proved once, and `Free` is reduced to three conditions on the carriers, so $SU(n)$ (`su n`) and $U(1)$ (`u1`, as 1×1 matrices) are obtained by filling in a presentation. Products (`prod`) and a list of factors (`ofFactors`) then give any group such as $SU(3) \times SU(2) \times U(1)$, written `[.SU 3, .SU 2, .U1]`.

Factors and morphisms. Charges are given one factor at a time, for example "a doublet of $SU(2)$ with hypercharge -3". A *factor* (`U1Factor`, `SUFactor`) records how the gauge jets project onto one $U(1)$ or $SU(n)$. A *morphism* (`Hom`) is a map between two data that respects all the maps above, and a factor of the target pulls back along it. The projections of a product are morphisms, which is how the factors of $SU(3) \times SU(2) \times U(1)$ are obtained from those of its pieces.

Conventions. The datum is always passed explicitly, never found by instance search, because the four types do not determine it. The layer imports nothing from the layers above it.

3.2 `MatterField/` and `GaugeBoson/`: the two kinds of field

Matter. `MatterField jets` has a complex value space V (free and finite), a Lorentz representation, the jet action `repJet` on `JetRing` $\otimes V$, required to be fibrewise and generated by the infinitesimal action `repAlgebra`, a mass weight and a statistic (PR #1989). Matrix representations and tables (`MatrixRep`) produce matter fields from charges one factor at a time and prove the infinitesimal-action condition once for all of them. The coefficients by which a jet moves the derivatives of a field live in `RepCoeff`, separate from the covariant derivative, so that field data can use them without importing the algebra.

The connection. The value space is CoVector $\otimes\, \mathfrak{g}$, real. The coefficients are the adjoint transport and the Maurer–Cartan coefficients (`GaugeCoeff`), with their composition laws: the linear part composes by a Leibniz sum, and the translation part is a cocycle. The connection is functorial in the gauge datum (§3.4).

Requirements from the classification. Keep the value-space representation distinct from the dual action on field symbols, including their conjugates. Parity and mass weight belong to the field; bases and particular fundamental, conjugate or adjoint representations are inputs to specific classifiers, not requirements on every field. Keep the conditions behind covariance visible: `GaugeLorentzCompatible` supplies Lorentz stability of covariant expressions, and `PureJetsActTrivially` makes their jet action factor through evaluation. Whether these become fields of `MatterField`, discharged once for matrix representations, or remain hypotheses on the field data is open (§5). The converse, that jet-invariant expressions lie in the covariant algebra, uses `Free` and is a separate result.

3.3 `FieldData/`: what a theory contains

The datum. The field data of a theory is the indexed family that exists today as `GaugeFieldData`: species types `FermionSpecies` and `BosonSpecies`, stored as fields of the structure with `Fintype` and `DecidableEq`, a `MatterField` for each species with the matching statistic, and the gauge field implicit. It is to be renamed `FieldData`, and the table layer's `LocalGaugeData.FieldData Γ Fields` then needs another name, such as `FieldTable`. The table compiles to it with species `Σ f, Fin (generations f)`, so names and generation counts stay where a Lagrangian is written, with Yukawa matrices indexed by `Fin 3 × Fin 3` per fermion type. All of `LocalFieldAlgebra/` and `LocalCovFieldAlgebra/` is built on it.

Presentation and meaning. The species are a choice. When a multiplet occurs more than once, as the three generations of each fermion or the two doublets of a two-Higgs-doublet model, splitting the theory into species is a choice of basis of the multiplicity space, and descriptions related by a change of that basis, a flavour rotation or a change of Higgs basis, are the same theory. What does not depend on the choice is the merged content: the *fermionic module* `T.FermionModule` $= \prod_i V_i$ and the *bosonic module* `T.BosonModule`, with their Lorentz and jet actions and their mass-weight grading.

These exist today and need no common mass weight. So the indexed family is the datum, because the algebra and the classification work species by species, and the merged modules are what morphisms see (§3.4). Both positions of 5 November were partly right: the morphisms belong to the merged content, the datum to the indexed family.

The merged alternative. On 5 November the thread chose instead one bosonic and one fermionic `MatterField`, merged by `MatterField.prod` and `pi`, because their inclusions and projections give embeddings directly. On 9 November Nathaneal found that `GaugeFieldData` is the indexed design and that the algebra is built on it, and reopened the question. As the datum, merged fields would need the algebra rebuilt and would forget the decomposition into multiplets that the classification uses. They also force one mass weight per statistic, which is mild: spinors have weight 3, scalars and vectors weight 2, and only fields such as supersymmetric auxiliary fields break it. The merged fields remain available as `bosonMatterField` and `fermionMatterField`, under a common-weight hypothesis.

Still to do.

- *Individual multiplets.* An accessor is the inclusion or projection of one species in the merged module, `inclBosonValue` and `projBosonValue` and their fermionic versions, which exist; as morphisms they form a retract (§3.4).
- *The empty case.* Empty species types; the pure gauge theory is the field data with no species.
- *Compatibility conditions.* `GaugeLorentzCompatible` and `PureJetsActTrivially` are stated species by species; the table layer proves the first for every compiled field.

Fields with an affine shift other than the connection (Stückelberg, axion), if ever needed, get their own family with an affine datum.

3.4 Morphisms: equivalence, embedding, projection

Joseph asked (4 November) that one model can be embedded in or projected onto another, and that two descriptions of one model can be shown equivalent. None of this is equality. `MatterField` carries a type V, so two isomorphic fields are never equal in Lean, whatever the shape of the field data; and a change of gauge group changes the type of the field data. Both are handled by morphisms, and the gauge side already has them: `LocalGaugeData.Hom` in `Physlib/`, with `id` and `comp`, and in `PhyslibAlpha/` the projections `Hom.fst`, `Hom.snd` and `Factor.comap`. The anomaly-cancellation library sets the house style in `ACCSystem.Hom`: the data of a map plus one commuting condition. What is missing is the matter side and the connection, three definitions.

Intertwiners, over one gauge datum. `MatterField.Hom M N` is a $\mathbb{C}$-linear map $V_M \to V_N$ commuting with the Lorentz action, the jet action (through `JetRing` $\otimes -$) and the gauge-algebra action, with equal mass weight and statistic. It was agreed on 6 November and Nathaneal has implemented it, with the quiver notation $M \longrightarrow N$ and no category instance, as Joseph preferred. With `id` and `comp` it composes; `Equiv` is an isomorphism; an embedding is an injective morphism and a projection a surjective one, packaged as a retract (ι, π) with $\pi \circ \iota = \mathrm{id}$. `prod` is a biproduct and `pi` a finite one: `inl`, `inr`, `fst`, `snd`, `proj i`, `single i` are morphisms, with `lift` and `desc`. The inclusion and projection of one species in the merged module are of this kind, and form a retract. The flavour group comes for free: a 3×3 matrix on `pi (fun _ : Fin 3 => M)` is an endomorphism, an invertible one an automorphism, the unitary ones being the flavour rotations once a hermitian structure is added, and "three copies of M are $M \times (M \times M)$" is an `Equiv`.

Restriction, along a morphism of gauge data. For a morphism of gauge data `φ : Hom jets jets'`, `MatterField.comap φ` sends a matter field over `jets'` to one over `jets`: the same V and Lorentz action, the jet action composed with `φ.groupJ`, the algebra action composed with `φ.lie`. The infinitesimal-action law is stated through `deriv`, `mulCoord`, `adjointJ`, `maurerCartan` and `evalLie`, and `Hom` commutes with each of these, so the law transports; those compatibility fields exist for this. It is contravariant, like `Factor.comap`, and compiling a pulled-back table row should agree with pulling back the compiled field, up to `Equiv`.

The connection, along a morphism of gauge data. The connection has no free data, so it needs no morphism type of its own, only functoriality. On values, `GaugeBoson.map φ` is $\mathrm{id} \otimes \varphi_{\mathfrak{g}}$ from $\mathrm{CoVector} \otimes \mathfrak{g}$ to $\mathrm{CoVector} \otimes \mathfrak{g}'$. The symbols of the gauge field are covectors, so on the jet algebras the induced map runs the other way, `LocalGaugeFieldAlgebra.comap φ` from the algebra of the $\mathfrak{g}'$ gauge field to that of the $\mathfrak{g}$ gauge field: an expression in the larger gauge field is evaluated on fields valued in the smaller algebra. It is equivariant, with G_J acting on the source through `φ.groupJ`, because `φ.lieJ_adjointJ` carries the adjoint transport, and `φ.lieJ_maurerCartan` the Maurer–Cartan coefficient, to those of `φ.groupJ U`; the Lorentz action is untouched. This is what the physics says: a Standard Model gauge field is an $SU(5)$ gauge field through $\mathfrak{g} \subset \mathfrak{g}'$, so any $SU(5)$ expression can be evaluated on it.

Morphisms of models. Over one gauge datum, a morphism $T \to T'$ is a pair of linear maps of the merged modules, commuting with every action and preserving the grading:

```
structure FieldData.Hom (T T' : FieldData jets) where
  fermion : T.FermionModule →ₗ[ℂ] T'.FermionModule
  boson   : T.BosonModule →ₗ[ℂ] T'.BosonModule
  -- commuting with the Lorentz, jet and gauge-algebra actions,
  -- and with the mass-weight scaling
```

Equivalently it is a matrix of `MatterField.Hom`s between species of equal weight. An equivalence is an invertible one, an embedding an injective one, a projection a surjective one; renaming rows, listing generations separately and changing basis are all equivalences. A bijection of species with an isomorphism on each is a special case, and too strict to be the definition: it would make a flavour rotation, or a change of Higgs basis, a different theory. The test is the automorphism group, which should be the global symmetries commuting with gauge and Lorentz transformations. For the Standard Model fermions, five pairwise distinct irreducible multiplets each occurring three times, Schur's lemma gives $GL(3,\mathbb{C})^5$, whose unitary part $U(3)^5$ is the flavour group once a hermitian structure is added (§3.5); species bijections give only permutations of generations and rescalings of each species.

Along a morphism of gauge data φ, a morphism from T to T' is a morphism from T to T' restricted along φ, species by species, the connection being carried by `GaugeBoson.map φ`. Along an isomorphism φ this gives equivalence up to an automorphism of the gauge datum, such as a rescaling of hypercharge or charge conjugation. Embedding the Standard Model in $SU(5)$ is an equivalence between the Standard Model data and the $SU(5)$ data restricted along the block embedding; projecting onto one factor is restriction along `Hom.fst`.

At the level of the algebras the direction reverses, because the symbols are covectors: a morphism $T \to T'$ induces an equivariant algebra map $J(T') \to J(T)$, from `JetComponentSpace.comap`, which exists, on each generator space and `LocalGaugeFieldAlgebra.comap` on the connection. For an embedding it restricts an expression to the sub-theory, setting the other fields to zero; for a projection it is injective, and its image is the sector of the sub-theory. This functoriality theorem is what makes the data-level notion trustworthy, and it is how sectors arise.

Branching of a gauge sector lives at the algebra level. Along $\varphi : \mathfrak{g} \hookrightarrow \mathfrak{g}'$ with $\mathfrak{g}' = \mathfrak{g} \oplus \mathfrak{m}$ as $\mathfrak{g}$-modules, the $\mathfrak{g}'$ gauge field seen over `jets` is not a gauge field of `jets`. It is the `jets` gauge field,

transforming affinely, together with a field valued in CoVector $\otimes$ $\mathfrak{m}$ transforming homogeneously, because $\omega(\varphi U)$ lands in $\mathfrak{g}$. These are the X and Y bosons, a bosonic *matter* field in $(3,2)_{-5} \oplus (\bar{3},2)_5$ with a Lorentz-vector label. A map of field data sends gauge to gauge, so this is not one; it is an isomorphism of algebras, the algebra of the $\mathfrak{g}'$ gauge field with G_J acting through φ against the algebra of the $\mathfrak{g}$ gauge field tensored with J of the coset field. It needs a vector Lorentz label and complexified real fields, neither of which the table has (§4), and it is the reason not to promise projections of models at the data level alone. This refines item 5 of §4: restriction along `Hom` branches every matter field; a gauge sector branches at the algebra.

Order. `MatterField.Hom` with its notation, in Nathaneal's coming PR; then `FieldData.Hom` on the merged modules, with the species inclusions and projections as retracts; then `comap` and the functoriality of the connection; then the functoriality of J. The rename of `GaugeFieldData` is independent of all of these. The open TODO on `LocalGaugeData.Hom`, a constructor from a Lie group homomorphism, is what the $SU(5)$ embedding needs.

3.5 `LocalFieldAlgebra/` and `LocalCovFieldAlgebra/`: the algebras

Each directory is named after the algebra it defines, `LocalFieldAlgebra` ($J(T)$) and `LocalCovFieldAlgebra`; today they are `LocalField/` and `LocalCovField/`.

$J(T)$ is built from the per-field generator spaces, bosonic generators commuting and fermionic ones anticommuting, with the connection's generators complexified. Its universal property says that every compatible assignment of the symbols to an associative complex algebra B extends to exactly one algebra map; B need not be commutative and the map need not be injective. Gauge and Lorentz actions, ordinary derivatives and mass weight are defined on generators. The current sectors use only the totals per category (fermion, gauge, scalar); sectors by sub-families follow from the maps induced by projections onto sub-theories (§3.4).

The covariant subalgebra is generated by the matter fields, their symmetrised covariant derivatives and the covariant derivatives of the field strength; on it the jets act through G_0. Each sector of the classification supplies a reduction, containment of its classified span in the source, and pointwise fixedness of that span; these combine without sector independence.

Conjugates and reality. The conjugate components already sit in each field's jet component space. Still to decide (§5): a real/complex flag per matter field, conjugate symbols generated and never listed, and a star structure on $J(T)$ that swaps a symbol with its conjugate and fixes real fields and the connection. A real Lagrangian is then a star-fixed invariant.

4 What a model sees

A model is a card and nothing else. The Standard Model card (`PhyslibAlpha/Particles/StandardModel/Basic.lean`) reads:

```
abbrev gauge : List FactorSpec := [.SU 3, .SU 2, .U1]
abbrev leptonDoublet : MatterFieldData factors := (.L, .singlet, .fund, -3)
abbrev higgs : MatterFieldData factors := (.scalar, .singlet, .fund, 3)
inductive Fields | q | l | u | d | e | H
def table : FieldData factors Fields | .q => (3, quarkDoublet) | ...
```

The fields of the model are abbreviations of the value spaces the card produces, and theorems are stated in the card and the generic vocabulary alone. This is the interface of model-building tools such as SARAH and Sym2Int: the gauge group and a list of matter fields with generations, the gauge fields never listed. To complete it:

1. real fields: a real-scalar label or a reality flag (`LorentzLabel` has only `L`, `R`, `scalar`, all complex), and a vector label, which a branched gauge sector needs (§3.4);
2. a gauge field and a field strength for each gauge factor, obtained from the factor projections, with names the card can use;
3. conjugates generated automatically, through the star structure and a user-facing conjugation;
4. writing a term from the card's symbols and checking its invariance (the realization of symbols in $J(T)$);
5. symmetry breaking (vacuum expectation values, mass eigenstates; restriction along a `Hom` branches each matter field, and a gauge sector branches at the algebra, §3.4) and global or discrete symmetries, including the flavour group acting on generations, which §3.4 gives as automorphisms of the field data.

Items 1–3 belong with the field-data work; item 4 is the realization PR; item 5 comes after the deadline.

5 Decisions and open questions

question	answer	status
One notion for matter and the connection, or two?	Two. A common interface exists only once the model is fixed (`Field T`).	agreed, 2–3 Nov
Connection explicit in the field data or implicit?	Implicit, from the gauge datum. Sums are disjoint unions of matter families.	agreed, 2 Nov
Shape of the field data	The indexed family of `GaugeFieldData`, renamed `FieldData`; a presentation, whose morphisms see only the merged modules (§3.3). One bosonic and one fermionic `MatterField`, chosen 5 Nov, would need the algebra rebuilt.	merged fields chosen 5 Nov; reopened 9 Nov
Maps between models	Morphisms, not equality: maps of the merged modules commuting with every action, equivalently matrices of `MatterField.Hom`s between species; restriction along `LocalGaugeData.Hom`; the connection functorial; algebra maps in the opposite direction (§3.4).	requirement 4 Nov; `MatterField.Hom` agreed 6 Nov; rest proposed
Branching of a gauge sector	At the algebra level, as the coset matter field; needs a vector label and real fields.	proposed
What happens to PR #2011	Drop the species type (agreed 3 Nov). Keep the coefficient extraction; move the gauge-field composition laws and the component form of the gauge law to `GaugeBoson/GaugeCoeff`.	type dropped; the rest proposed
Real or complex tower	One complex tower with a star, real objects being its star-fixed parts; the real connection algebra stays until step 5 of §6.	proposed
Sectors	Images of the algebra maps induced by projections onto sub-theories. A grading by sub-families waits until a proof needs it.	proposed
Conjugates and reality	Reality flag, generated conjugates and a star, as in §3.5.	open
Compatibility conditions	Fields of `MatterField` or hypotheses on the field data.	open
Names	`FieldData` for the field data (agreed 3 Nov); directory `LocalFieldAlgebra/` for $J(T)$ (agreed 3 Nov); `LocalCovFieldAlgebra/` by the same rule and `Field T`; the table's `LocalGaugeData.FieldData` renamed, for example `FieldTable`.	partly agreed
Order of work	Clean up and move layer by layer in the order of §3; once `FieldData` is in, the later layers proceed in parallel.	agreed, 3 Nov

The open questions on conjugates and on the compatibility conditions should be settled before the realization of symbols in $J(T)$.

6 Status of the tree

Measured on Physlib `master` on 3 November 2026 (directories under `PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/` unless stated). There are no `sorry`s.

layer	now	against the design
`LocalGaugeData/`	19 files, about 6 500 lines	Done and imports nothing above it; only `Basic` is in `Physlib/`. About 3 100 lines of $SU(n)$ invariant theory (`SU/Invariants`, `GellMann`, `Adjoint`, `TensorSpecies`) belong to the factor reductions of `LocalCovFieldAlgebra/`.
`MatterField/`	15 files, about 5 400 lines	Structure with statistic, `TransformsIn`, covariant derivative, `MatrixRep` and tables, `Prod`, `Pi`; conjugate components present. No reality flag; compatibility conditions are separate definitions.
`GaugeBoson/`	17 files, about 6 900 lines	`Basic` (216 lines) as intended. The rest is the real connection tower: `LocalGauge` 2 800, `LocalGaugeCov` 600, `Realization` 3 200. `GaugeCoeff` exists only in PR #2011.
`GaugeFieldData/`	7 files, about 2 000 lines	The indexed field data, gauge field implicit, with the merged modules and their actions; to be renamed `FieldData` (§3.3). No `Field T` and no morphisms: `MatterField.Hom` is in Nathaneal's coming PR, and `LocalGaugeData.Hom` is the tree's only morphism.
`LocalField/`	13 files, about 4 600 lines	$J(T)$ with universal property, actions, derivatives, mass weight, realizations, covariant derivatives including conjugates. Sectors only by category totals, built by hand; no functoriality in the index types; no star.
`LocalCovField/`	6 files, about 2 200 lines	Covariant subalgebra, sectors, realizations.
`Mathematics/InvariantReduction`, `StandardModel/InvariantReduction`	537 and 309 lines	The mechanism exists; the combined family and the factor reductions are still stated for the Standard Model only.
Model card (`Particles/StandardModel/Basic`)	148 lines	The intended interface, used only by `Model/Consistency` and `Model/LeptonDoublet`.

The headline result, `mem_massWeightSubmoduleLE_eight_and_invariant_iff_lagrangian` in `StandardModel/JetAlgebra/AlgebraRealization`, classifies the invariants of mass weight at most eight. It is proved, but on the hand-built field data (`StandardModel.fieldData`, fifteen fermion species and the Higgs) rather than on the card, and it classifies invariant polynomials with no quotient by total derivatives, so the word *lagrangian* in its name says more than it proves. The Standard Model directory is about 55 000 lines.

So the remaining work before the deadline is mostly migration: restating that result through the card and the generic field data, not new proofs.

In order:

1. **Done.** `LocalGaugeData/` imports nothing above it (PR #1415 and the `LocalGaugeData/` PRs).
2. **Clean-up layer by layer** (agreed 3 November). `LocalGaugeData/` moves to `Physlib/` except the $SU(n)$ invariant files, which move to the classification. Then `MatterField/` and `GaugeBoson/Basic` with `GaugeCoeff`.
3. **Field data.** Rename `GaugeFieldData` to `FieldData`; Nathaneal's PR with `MatterField.Hom`; morphisms of field data on the merged modules (§3.4), `Field T`, conjugates and compatibility, then the realization of the symbols in $J(T)$. After this the later layers proceed in parallel.
4. **Sectors and the card.** Rename the algebra directories, derive the sectors from maps of index types, and migrate the Standard Model from the hand-built field data to the card.

5. **After the deadline.** Replace the real connection tower by the field data with no matter, last, because the Standard Model gauge-sector classification is built on it; add reality and a total-derivative derivation commuting with the gauge action, so that invariants can be classified modulo total derivatives.

Steps 2–4 are the target for early December, with the end of the semester as the hard deadline.

7 Progress

`LocalGaugeData/` (September 2026, PR #1415)

The layer imported matter in two ways; both are gone. The factor types `U1Factor`, `SUFactor`, `Factor`, `Factors` moved down into `Factor.lean`, since their fields mention only gauge jets, the gauge algebra and the jet ring. `TransformsIn` and `InfinitesimalAction`, statements about a representation on `JetRing` $\otimes$ V, moved up to `MatterField/`. The duplication between `U1.lean` and `SU/Basic.lean` was removed by `MatrixJets.lean`, a presentation of gauge jets by matrices of jets from which the laws, faithfulness and the canonical `SU`-type factor are proved once. `MatrixJets.free` (Joseph, 25 September) reduces `Free` to three conditions on the carriers, and the Standard Model freeness proof became an instance, about 400 lines shorter. The layer is eleven files, about 3 600 lines, importing nothing outside itself: `Basic`, `AdjointCoeff`, `MaurerCartan`, `Truncation`, `Factor`, `MatrixJets`, `SU/Algebra`, `SU/Basic`, `U1`, `Prod`, `OfFactors`.

Not done: `U1.lean` and `SU/Basic.lean` still define their structure maps by hand. Defining the carriers as subtypes of matrices would change the types the Standard Model files unfold to, so it waits for the hand-built Standard Model code to go.

Matter fields and species (PRs #1989 and #2011, October–November 2026)

PR #1989 (merged 30 October) added `statistic : FieldStatistic` to `MatterField` and requires the fermionic and bosonic entries of the field data to match; merging fields now needs a common statistic as well as a common mass weight. PR #2011 (opened 1 November) introduced a species type with case-defined coefficients and gauge, weight, derivative and Lorentz laws, and moved the coefficients into `MatterField/RepCoeff` and `GaugeBoson/GaugeCoeff`. After discussion on 2 November the species type was dropped, confirmed on 3 November, and the PR is to be reworked as in §5.

Naming and plan (3 November 2026)

The thread agreed to name the field data `FieldData`; to name the directory holding $J(T)$ after its algebra, `LocalFieldAlgebra/`, as Joseph suggested; and to clean up the existing code layer by layer in the order of the diagram. Nathaneal is preparing the `FieldData` PR.

Field data and morphisms (4–9 November 2026)

On 4 November Nathaneal proposed a named field data, a family `Name → MatterField` with multiplicities, and Joseph objected that it is hard to say when two such data are equivalent, since the names differ; he added that he also wants embeddings of one model into another, and projections. Nathaneal worked both shapes through on 5 November: the named family gets equivalence from a bijection of copies and gets embedding and projection by zero-filling and forgetting entries, while the pair gets all three from `MatterField.prod` directly. He and Joseph chose the pair, to be tried first and refactored if needed; by the evening it was connected to `LocalFieldAlgebra`, with `LocalCovFieldAlgebra` next and a PR to follow. The morphism design of §3.4 was written in response on 6 November, after checking what the tree has: `LocalGaugeData.Hom` with `Hom.fst`, `Hom.snd` and `Factor.comap`; the table layer's

`FieldData Γ Fields`, which is the named proposal at the table level; `ACCSystem.Hom`; and no morphism of matter fields.

On 6 November Jinzheng proposed `MatterField.Hom` on the thread; Joseph and Nathaneal agreed, and Nathaneal implemented it with the quiver notation $M \longrightarrow N$, avoiding a category instance as Joseph preferred. On 9 November, implementing the merged structure, Nathaneal found that `GaugeFieldData` is the indexed design, split by statistic, and that all of `LocalFieldAlgebra` is built on it; he decided to open a smaller PR with the new structure, the notation and `MatterField.Hom`, so that this could be discussed. This note was revised the same day: the indexed family stays the datum and morphisms are defined on the merged modules (§3.3, §3.4).

Luca Mantani joined the thread on 5 November, intending to work towards the SMEFT, and on 6 November chose to start with the Higgs convention. Joseph agreed that the old convention placing the Higgs vacuum expectation value in the first component of the doublet should be changed to the standard one; Nathaneal pointed at the copy-pasted fermion code and at upstreaming as places to start. Higher mass dimension and the quotient by total derivatives and equations of motion are the gaps the SMEFT would need closed.